SOFTWARE ENGINEERING — LAB 7 DELIVERABLE

Fuzzing for Input Validation Bugs

Property-Based Testing with Hypothesis • Repository: `L6_L7_08_ski_resort_pass`

1. Summary of Discovered Edge Cases & Defensive Fixes

Using the Hypothesis property-based testing library (`@given(st.text() | st.none())`), automated fuzzing was executed against all three functions in `processor.py` . The fuzzer exposed unhandled runtime exceptions when processing `None` , empty strings, and malformed delimiters.

Function	Fuzz Inputs & Exceptions	Defensive Fix Applied
`parse_season_pass_validity`	`s=None` (`AttributeError`), `s='0'` (`IndexError`), `s=''` (`ValueError`)	Verified `isinstance(date_str, str)` , enforced exact 3-part `'/'` split, and wrapped `int()` conversions in `try-except ValueError` .
`compute_trail_gradient`	`s=None` (`AttributeError`), `s=''` (`ValueError`), zero distance (`ZeroDivisionError`)	Verified string type, checked 2-part `':'` split, caught `ValueError` on `float()` parsing, and guarded against `distance == 0.0` .
`calculate_lift_access_tier`	`s=None` (`AttributeError`), `s=''` / unknown codes (`KeyError`)	Added `isinstance(zone_code, str)` validation and replaced direct dictionary indexing with safe `zone_rates.get(prefix, 0.0)` .

2. Step 2 — Initial Execution of Buggy Application

Running the provided `processor.py` script with well-formed inputs (`2026/12/31` , `250:1000` , and `BEG`) executes normally without revealing underlying input validation vulnerabilities.

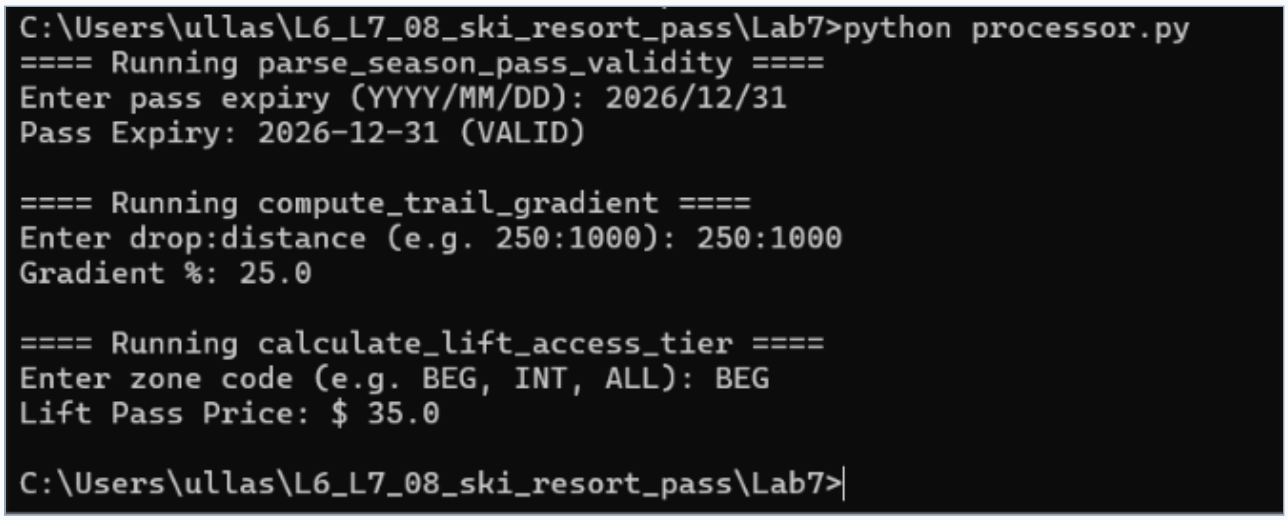


Figure 1: Output of `python processor.py` prior to fuzz testing and input hardening.

3. Step 4 — Hypothesis Fuzz Testing Output (Failing Cases)

Executing `python -m pytest test_processor.py` generated edge-case inputs that crashed all three functions in the unhardened module (`3 failed in 2.38s`).



Figure 2a: Pytest and Hypothesis failure report for `test_parse_season_pass_validity_no_crash` .



Figure 2b: Pytest and Hypothesis failure report for `test_compute_trail_gradient_safe` and `test_calculate_lift_access_tier_safe`.

4. Step 5 — Execution of Hardened Application

After implementing type validation, structural delimiter checks, exception handling, and safe lookups in `processor.py`, the script was re-executed to verify expected functional behavior.

```
C:\Users\ullas\L6_L7_08_ski_resort_pass\Lab7>python processor.py
==== Running parse_season_pass_validity ====
Enter pass expiry (YYYY/MM/DD): 2026/12/31
Pass Expiry: 2026-12-31 (VALID)

==== Running compute_trail_gradient ====
Enter drop:distance (e.g. 250:1000): 250:1000
Gradient %: 25.0

==== Running calculate_lift_access_tier ====
Enter zone code (e.g. BEG, INT, ALL): BEG
Lift Pass Price: $ 35.0
```

Figure 3: Output of `python processor.py` after applying defensive input validation fixes.

5. Step 6 — Re-Running Fuzz Tests on Fixed Code (Passing Cases)

Re-running `python -m pytest test_processor.py` confirmed that all three functions gracefully handle arbitrary Unicode strings and `None` inputs without raising unhandled exceptions.

```
C:\Users\ullas\L6_L7_08_ski_resort_pass\Lab7>python -m pytest test_processor.py
============================= test session starts =============================
platform win32 -- Python 3.13.15, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\ullas\L6_L7_08_ski_resort_pass\Lab7
plugins: platformdirs-4.14.5, hypothesis-6.170.15
collected 3 items

test_processor.py ...                                                    [100%]

============================== 3 passed in 0.60s ==============================
```

Figure 4: Final pytest verification showing all 3 Hypothesis fuzz test suites passing (`3 passed in 0.60s`).